

Implementation of FTP Server using C on Linux

Name: Harsh Soni

PRN No: 240344230036

Mentor: Mr. Sohail Inamdar

Organization: Sunbeam Infotech, Pune

Date: September 05, 2025

**Sunbeam Infotech, Pune
2025**

Table of Content

1. Abstract
2. Introduction
3. Brief History
4. Purpose
5. System Requirements
6. Implementation
7. Testing and Results
8. Future scope
9. Challenges and Solutions
10. Conclusion

1.0 Abstract

File Transfer Protocol (FTP) is a standard network protocol used for transferring files between client and server across computer networks. This project aims to implement a custom FTP server using the C programming language on the Linux platform. The key modules developed include the core server setup, client handling routines, robust user authentication, and support for FTP file transfer commands. The project demonstrates handling concurrent clients, secure authenticated sessions, and enables file upload/download operations compliant with RFC 959. The developed server achieves functional parity with basic FTP needs and offers a foundation for further enhancements such as secure transfer. The outcomes highlight proficiency in Linux socket programming and protocol implementation, providing a valuable learning resource for networking education and practical applications.

2.0 Introduction

FTP (File Transfer Protocol) is a widely-used protocol designed for transferring files between clients and servers in a networked environment. FTP operates over TCP/IP and offers capabilities for uploading, downloading, listing directory contents, and user authentication. In real-world scenarios, FTP facilitates website maintenance, sharing software distributions, and backup operations. The objective of this project is to build a minimal FTP server using C on Linux, supporting essential FTP operations (upload/download, user authentication, directory listing), and concurrent handling of multiple clients, providing practical experience in network programming.

3.0 Brief History

FTP was first proposed in 1971 and was developed for implementation on hosts at M.I.T (Massachusetts Institute of Technology, RFC 114). It has been evolved over the years since then.

The RFC 114 has been upgraded into RFC 172 that provides a user-level oriented protocol for file transfer between host computers. A revision of this as RFC 265, restated FTP for additional review, while RFC 281 suggested further changes. The use of a "Set Data Type" transaction was then proposed in RFC 294 in January 1982. Then the RFC 354 obsolete both RFC 264 and RFC 265. The File Transfer Protocol was now defined as a protocol for file

transfer between Hosts on the ARPANET, with the primary function of FTP defined as transferring files efficiently and reliably among hosts and allowing the convenient use of remote file storage capabilities.

Changes have been made to the proposal and RFC 385 further commented errors on the previous version of FTP and emphasis the points, and add some additions to the proposal. Not long after that more features has been added whereas, RFC 414 provides a status report on the working server and user FTPs. RFC 430, issued in 1973 and presented further comments on FTP. Finally, official FTP document was published as RFC 454.

By July 1973, considerable changes from the last versions of FTP were made, but the general structure remained the same. RFC 542 was published as a new official specification of reflect these changes.

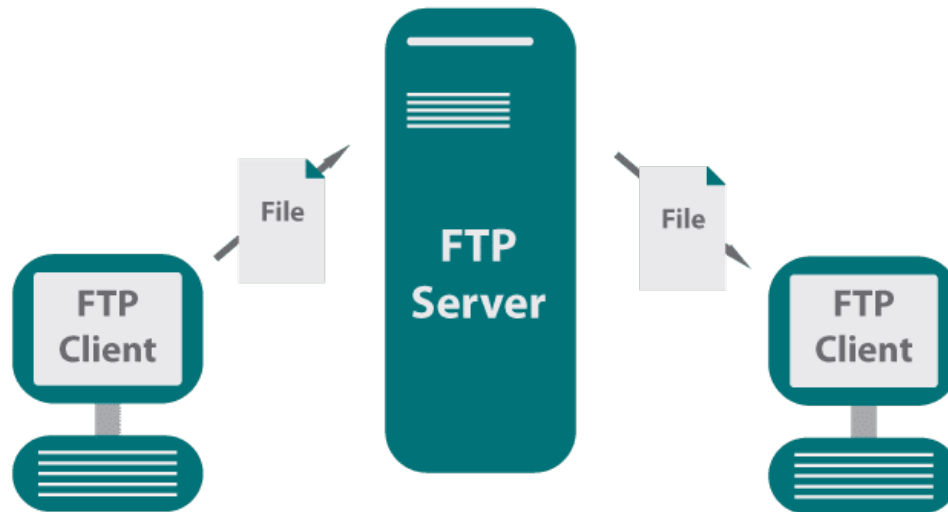
In 1974, RFC 607 and RFC 614 continued comments on FTP. RFC 624 proposed further design changes and minor modifications. In year 1975, RFC 686 entitled Leaving Well Enough Alone has been published to discussed the differences between all of the early and later versions of FTP. RFC 691 presented a minor revision of RFC 686, regarding the subject of print files.

Motivated by the transition from the NCP to the TCP as the underlying protocol, a phoenix was born out of all of the above efforts in RFC 765 as the specification of FTP for use on TCP.

4.0 Purpose

The objective of FTP is to promote files sharing (computer programs and/or data), to encourage indirect or implicit (via programs) use of remote computers, to shield a user from variations in file storage systems among hosts and to transfer data reliably and efficiently. FTP, though usable directly by a user at a terminal, is designed mainly for use by programs.

There are two machines involved in an ftp transaction, a client machine, which is sometimes called the local host, and a server machine, which is sometimes called the remote host. The client is always the machine that initiates the transfer. During an ftp session, it is extremely important to keep track of which machine is the client and which is the server, because this will determine whether you use a get command or a put command to move files. The get command is always used to copy files from the server to the client, and the put command is used to copy files from the client to the server.



Another use of FTP is to backup files. We should always make backup copies of our files in preventing it from damage or lost. People often use ftp to send their documents to others disk or server (via ftp).

5.0 System Requirements

Hardware: Standard PC/laptop with minimum 4 GB RAM, any modern CPU.

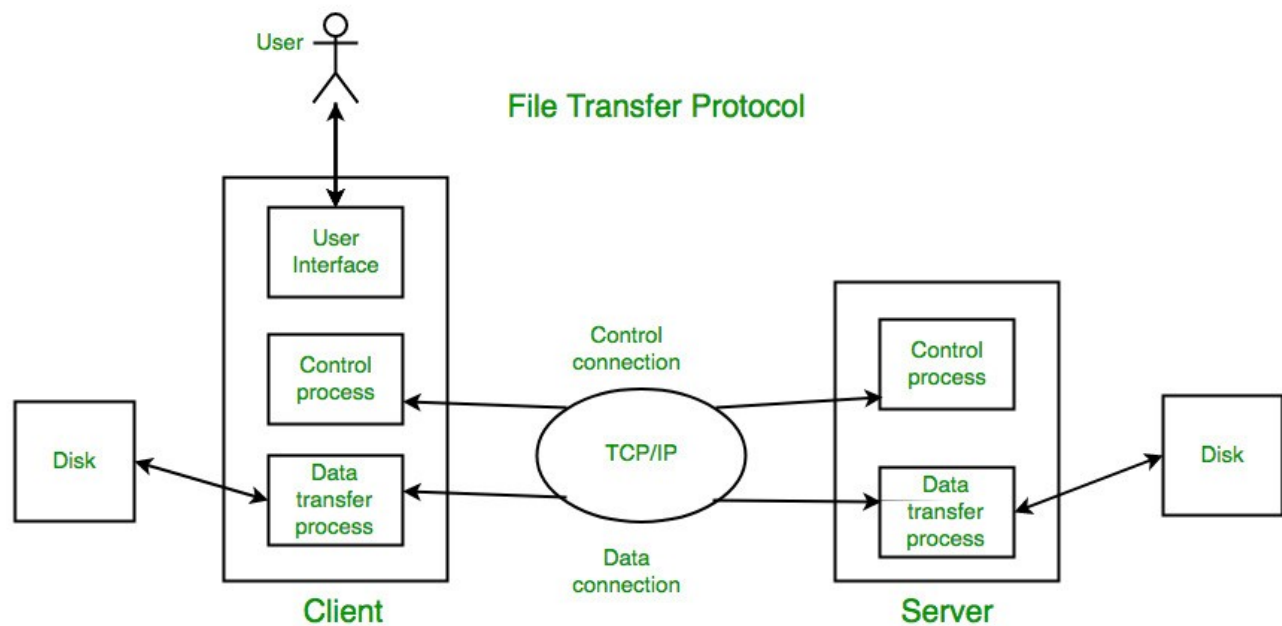
Software:

- Ubuntu 22.04
- GCC compiler (gcc)
- Socket and thread libraries (sys/socket.h, pthread)
- Wireshark for packet capture and debugging.

6.0 Implementation

Implementing basic FTP sever:-

Term	Meaning
IP Address	Like your home address on the internet (e.g., 192.168.1.1)
Port	Like a door number at your home (e.g., 8080)
Server	A program that waits for requests (e.g., a restaurant)
Client	A program that sends a request (e.g., a customer)
Socket	A connection pipe between client and server



Now Break Down Socket Programming into 5 Steps (Just Concepts)

Step What You Do Example

- | | | |
|---|-------------------|--|
| 1 | Create a socket | "Plug in the phone" |
| 2 | Bind to IP & Port | "Give the phone a number" |
| 3 | Listen | "Wait for a call" |
| 4 | Accept | "Pick up the phone when someone calls" |
| 5 | Communicate | "Talk: Send/Receive message" |

Server Side:

1. socket()
2. bind()
3. listen()
4. accept()
5. recv() / send()
6. close()

Client Side:

1. socket()
2. connect()
3. send() / recv()
4. close()

C Functions:

Function	What It Does
socket()	Creates a socket
bind()	Assigns IP address & port to the socket
listen()	Waits for clients
accept()	Accepts client connection

<code>recv()</code>	Receives message from client
<code>send()</code>	Sends message to client
<code>close()</code>	Closes the socket

Step 1: `socket()` — Creating the Socket

What does it do?

It creates a virtual pipe to connect two devices.

Step 2: `bind()` — Assign IP Address & Port to Your Socket

What does it do?

It assigns your socket a specific IP address and port number so other devices know where to connect.

Analogy:

You got a phone (`socket()`), now you're getting a phone number (`bind()`), so people can call you.

Step 3: `listen()` — Tell the Socket to Start Listening

What does it do?

Once the server has a socket and a port (`bind()`), you tell it:

"Now start listening — wait for incoming calls (clients)."

Analogy:

You got a phone (`socket()`), got a number (`bind()`), and now you're saying:

"I'm ready to take calls" — that's what `listen()` does.

Step 4: `accept()` — Accept a Client Connection

What does it do?

The server has already started listening.

Now it's saying:

"When a client connects, I will accept the connection and talk to it."

Analogy:

You're a restaurant owner. You've opened your door for customers (`listen()`), and now when a customer walks in, you greet and seat them (`accept()`).

Step 5: Communication – `recv()` and `send()`

What happens now?

Once the server accepts the client, a new socket (`client_fd`) is created — a direct pipe between the server and client.

Now the server can:

`recv()` → receive a message from the client

`send()` → send a response back

Summary of All 5 Steps

Function	Purpose
<code>socket()</code>	Create a socket
<code>bind()</code>	Assign IP + port
<code>listen()</code>	Start waiting for connections
<code>accept()</code>	Accept client request
<code>recv()</code> / <code>send()</code>	Exchange messages

Closes the connection.

7.0 Testing & Results

A. Test Cases Performed

7.1. Login with Valid / Invalid Credentials

- **Valid Credentials:** Confirm that the server accepts correct usernames and passwords, returning standard FTP reply codes such as 230 User logged in, proceed
- **Invalid Credentials:** Verify that the server rejects incorrect login attempts, producing an error like 530 Not logged in.

- **Security and Configuration:** Ensure your server is configured to handle authentication securely—double-check credential verification, firewall settings, and server access.

7.2. File Upload & Download

- **Upload Testing:**
 - Attempt to upload various file types (text vs binary) and sizes.
 - Verify transfer completion and data integrity via file size comparisons and checksum validation.
- **Download Testing:**
 - Download files and verify their integrity and content.
- **Protocol Modes:**
 - Ensure both ASCII and Binary modes are handled to avoid corruption.
- **Sample Functional Test Template** (adapted from sample FTP test case collections)
 - Verify transfer initiation.
 - Confirm mode availability (ASCII/Binary).
 - Ensure uploaded file remains intact.
 - Validate progress/log output.

7.3. LIST Directory Contents

- Testing the LIST or NLST command:
 - Confirm directory contents are correctly listed on the client.
 - Validate format consistency and readability.
 - Note server responses, e.g., 150 File status okay; about to open data connection, followed by 226 Closing data connection.

7.4. Multiple Clients Connected Simultaneously

- Use parallel client sessions (via forks, threads, or multiple terminal windows).
 - Ensure:
 - All clients maintain independent sessions.
 - Commands and file transfers do not interfere with each other.
 - Server remains stable under concurrent load.
-

B. Screenshots to Include

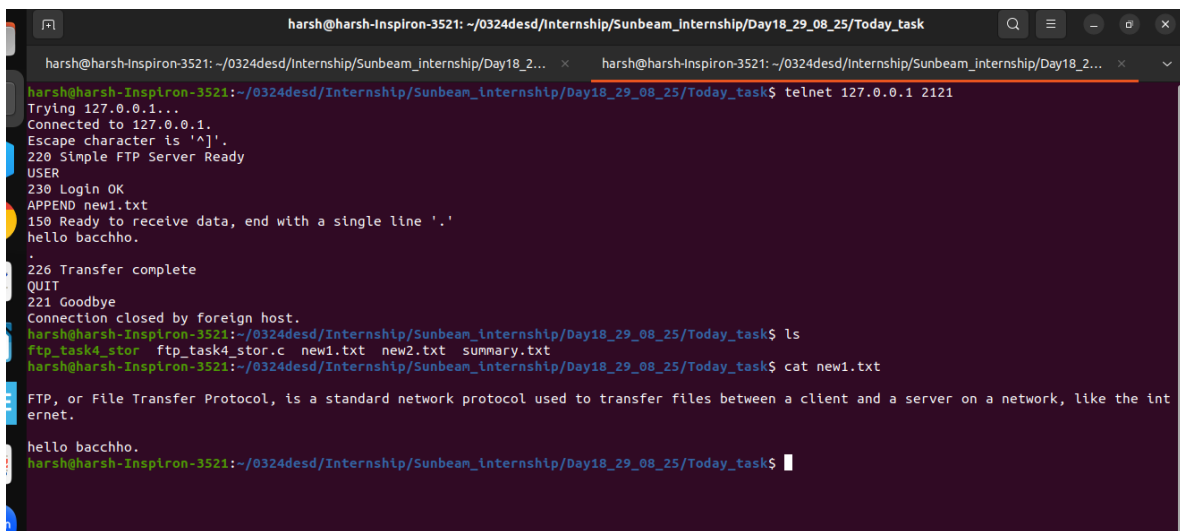
Consider including the following screenshots to visually document your testing:

1. Server Terminal at Startup

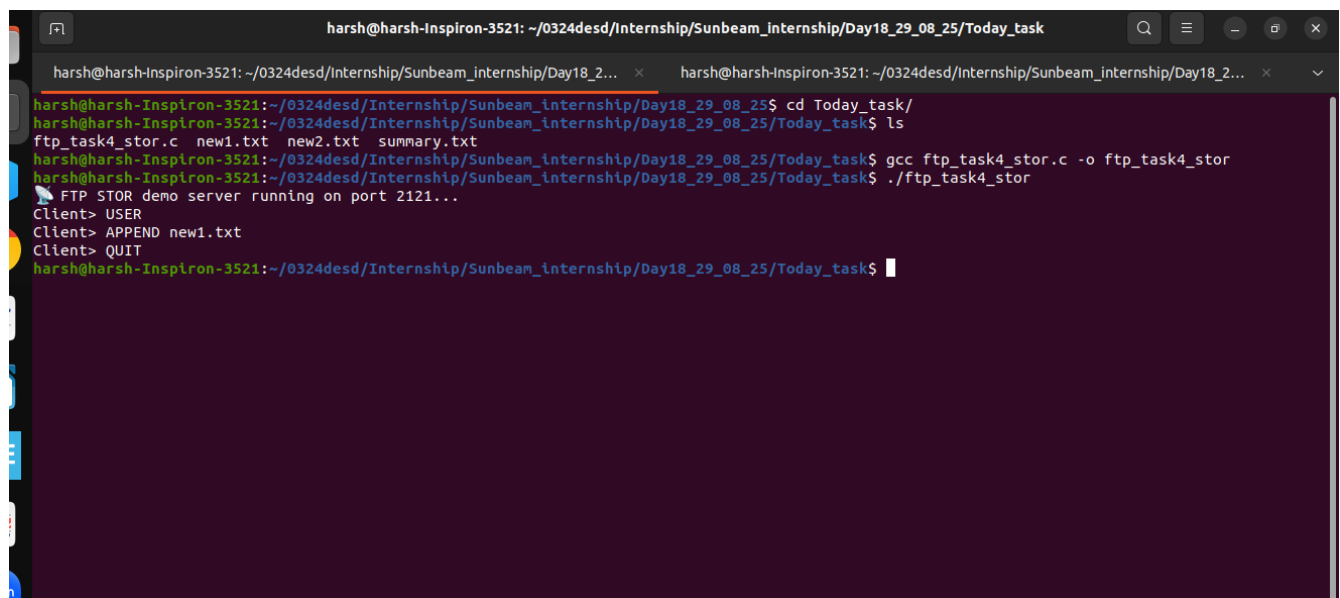
- Display the FTP server initializing, listening on control port.

2. Client Interactions

- *Successful Login:*
- *Invalid Login Attempt:*



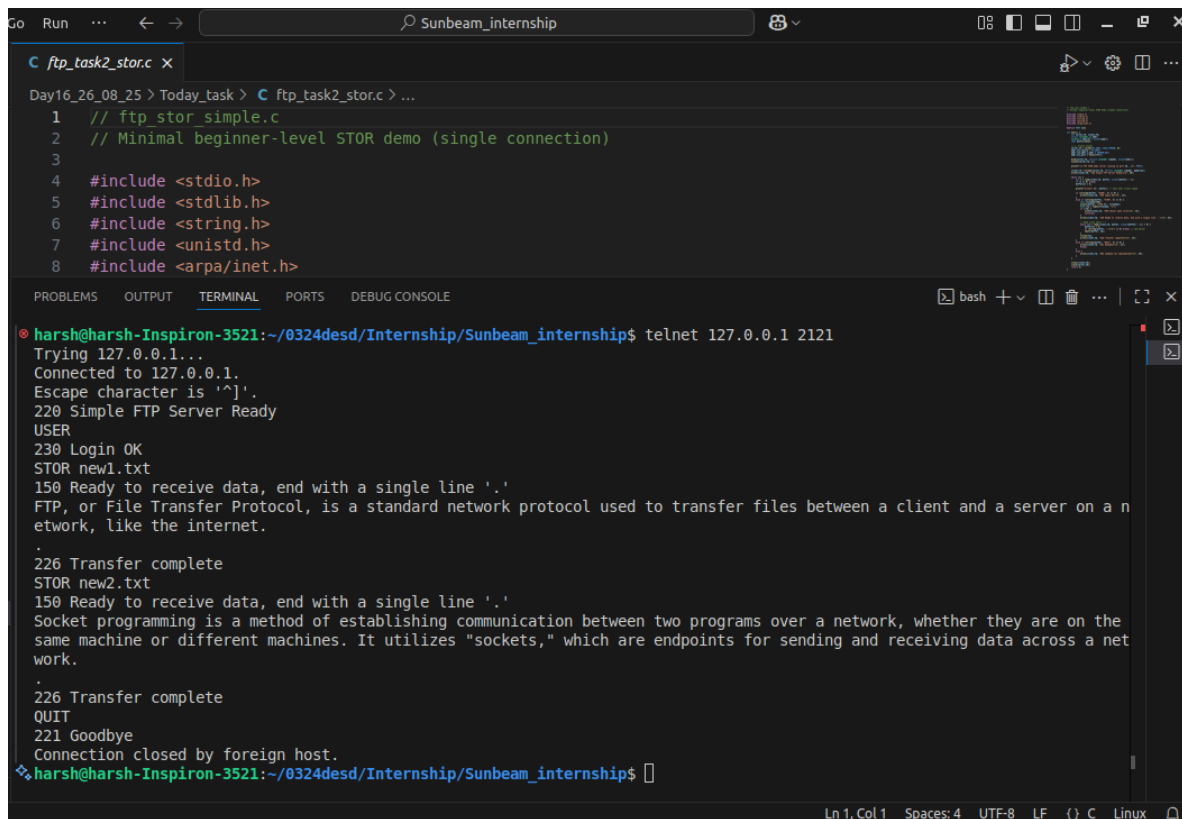
```
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task$ telnet 127.0.0.1 2121
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^['.
220 Simple FTP Server Ready
USER
230 Login OK
APPEND new1.txt
150 Ready to receive data, end with a single line '.'
hello bacchho.
.
226 Transfer complete
QUIT
221 Goodbye
Connection closed by foreign host.
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task$ ls
ftp_task4_stor  ftp_task4_stor.c  new1.txt  new2.txt  summary.txt
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task$ cat new1.txt
FTP, or File Transfer Protocol, is a standard network protocol used to transfer files between a client and a server on a network, like the internet.
hello bacchho.
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task$
```



```
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25$ cd Today_task/
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task$ ls
ftp_task4_stor.c  new1.txt  new2.txt  summary.txt
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task$ gcc ftp_task4_stor.c -o ftp_task4_stor
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task$ ./ftp_task4_stor
FTP STOR demo server running on port 2121...
Client> USER
Client> APPEND new1.txt
Client> QUIT
harsh@harsh-Inspiron-3521: ~/0324desd/Internship/Sunbeam_internship/Day18_29_08_25/Today_task$
```

3. File Upload & Download Screens

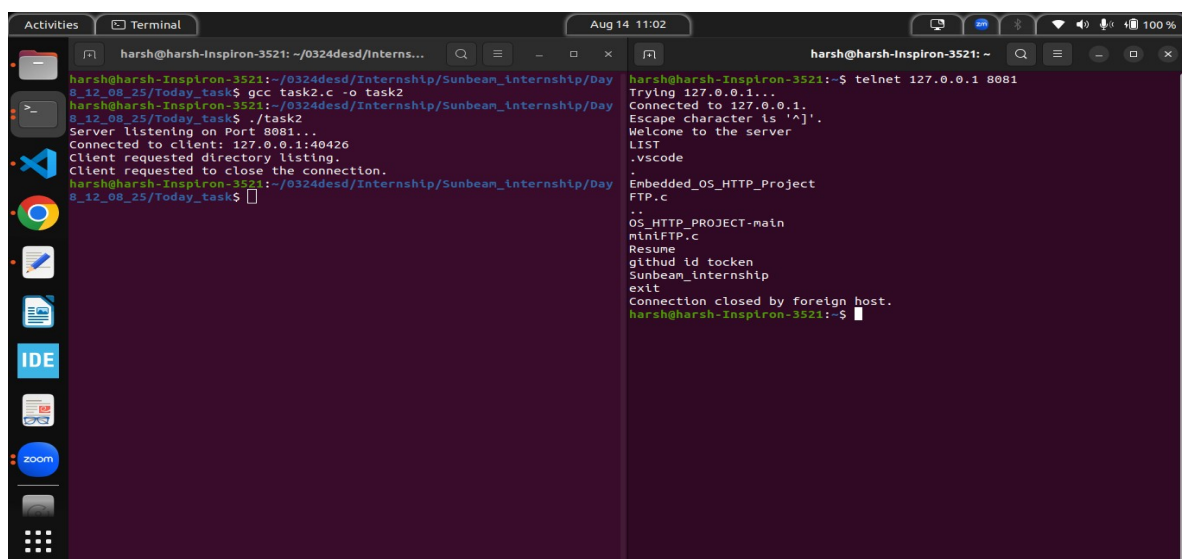
- Demonstrate uploading a file with command (STOR filename) and successful acknowledgment.



```
Go Run ... < -> Sunbeam_internship
C ftp_task2_stor.c x
Day16_26_08_25 > Today_task > C ftp_task2_stor.c > ...
1 // ftp_stor simple.c
2 // Minimal beginner-level STOR demo (single connection)
3
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <unistd.h>
8 #include <arpa/inet.h>
...
harsh@harsh-Inspiron-3521:~/0324desd/Internship/Sunbeam_internship$ telnet 127.0.0.1 2121
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
220 Simple FTP Server Ready
USER
230 Login OK
STOR new1.txt
150 Ready to receive data, end with a single line '.'
FTP, or File Transfer Protocol, is a standard network protocol used to transfer files between a client and a server on a network, like the internet.
.
226 Transfer complete
STOR new2.txt
150 Ready to receive data, end with a single line '.'
Socket programming is a method of establishing communication between two programs over a network, whether they are on the same machine or different machines. It utilizes "sockets," which are endpoints for sending and receiving data across a network.
.
226 Transfer complete
QUIT
221 Goodbye
Connection closed by foreign host.
harsh@harsh-Inspiron-3521:~/0324desd/Internship/Sunbeam_internship$
```

4. Directory Listing

- Show output of LIST command with server response codes and directory content.



```
Activities Terminal Aug 14 11:02
harsh@harsh-Inspiron-3521: ~/0324desd/Interns...
harsh@harsh-Inspiron-3521:~/0324desd/Internship/Sunbeam_internship/Day
0_12_08_25/Today_task$ gcc task2.c -o task2
harsh@harsh-Inspiron-3521:~/0324desd/Internship/Sunbeam_internship/Day
0_12_08_25/Today_task$ ./task2
Server listening on Port 8081...
Connected to client: 127.0.0.1:40426
Client requested directory listing.
Client requested to close the connection.
harsh@harsh-Inspiron-3521:~/0324desd/Internship/Sunbeam_internship/Day
0_12_08_25/Today_task$
harsh@harsh-Inspiron-3521:~$ telnet 127.0.0.1 8081
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
Welcome to the server
LIST
.vscodes
Embedded_OS_HTTP_Project
FTP.c
..
OS_HTTP_PROJECT-main
miniFTP.c
Resume
github id token
Sunbeam_Internship
exit
Connection closed by foreign host.
harsh@harsh-Inspiron-3521:~$
```

8.0 Future Scope

FTP will be enhanced in next future whereas the SSH Communications Security Corp. a world-leading provider of enterprise security solutions and end-to-end communications security has announced for the future, FTP or Enhanced File Transfer will be more secure (SFTP-Secure File Transfer Protocol) especially in confidentiality, integrity, and authentication for both interactive and unattended file transfers in multi platform system. Other than that, FTP also will be upgraded whereby some features will be added in it such as:

1. Transparent FTP-SFTP conversions that simplify FTP replacement projects by eliminating the need to modify existing scripted file transfers. Meaning that in upgrading the FTP to SFTP, there is no need for users to uninstall and reinstall the software whereas it is automatically upgrade the previous script of FTP.
2. Automatic guaranteed delivery with configurable alarms to ensure completion of critical file transfers. It is said that this enhanced FTP will guaranteed deliver files to server or vice versa and if the files were not completed send, an alarm will tell them that there are some files haven't been sent.

9.0 Challenges and Solutions

FTP Security Challenges

FTP was not designed with security in mind, leading to several vulnerabilities:

- Lack of Encryption:** This is because data such as usernames and passwords are transmitted without encryption and hence easily vulnerable to different attacks.
- Vulnerabilities to Attacks:** FTP transmissions are vulnerable to attacks such as spoofing, sniffing, brute force , and the likes are slow attacks. They suggested that the data can be intercepted and changed easily by hackers .
- Clear-Text Passwords:** Unfortunately, FTP client authentication employs non encrypted passwords, which makes them vulnerable to hacking.

Socket errors (port already in use, connection reset)

```
46
47     addr_size = sizeof(client_addr);
48     client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_size);
..
PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE
bash - Today_Task + - [ ] ... [ ] X
harsh@harsh-Inspiron-3521:~/0324desd/Internship/Sunbeam_internship/Day5_07_08_25/Today_Task$ gcc tcp_server.c -o tcp_server
harsh@harsh-Inspiron-3521:~/0324desd/Internship/Sunbeam_internship/Day5_07_08_25/Today_Task$ ./tcp_server
Bind failed!
: Address already in use
harsh@harsh-Inspiron-3521:~/0324desd/Internship/Sunbeam_internship/Day5_07_08_25/Today_Task$
```

Solutions

This allows your server to reuse the port immediately after it's closed, avoiding the "Address already in use" error during development.

```
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}
```

10.0 Conclusion

The fast developments of information technology made any resources are spread all over the world. Peoples can get any resources from any where as long as they have a computer and internet connection. The internet technology made it is easy for us to download files from one site to our computer and we also can upload files form our computer to the other site.

As being discussed in this research paper, the things that made files transfer become easy over the internet network is the implements of FTP (File Transfer Protocol). In the early age of file transmitting over internet, we only use e-mail attachment to transferring files. However e-mail attachment has some limitation such as limit the types of files and sizes of files we can transter. Thus, to overcome such issues we use FTP. FIP enabled us to transfer files without any limits of file types and size, which can not be done through e-mail attachment.

However, there are also other things that we need to consider regarding FTP technology such as security and privilege of accessing other site. Some FTP sites may provide public access, which means anyone can connect to the FTP site. But, there is also some FTP which restricts us to access to FTP sites and needs us to have an accounts with the FTP site's owner or sponsor to access to their sites.

Besides, in this research paper also we have discussed the prospect for the future of FTP whereas there are some other technologies of FTP to be implements. It is aims to upgrade the current services offer by FTP to make transferring files become more secure. In example are Enhanced File Transfer (EFT) and SFTP (Secure File Transfer Protocol) which will focus especially in confidentiality, integrity, and authentication for both interactive and unattended file transfers in multi platform system.

At the end, we hope this research paper will give a good explanation on how FTP works, how it was used, and how the future FTP may help us in transmitting files over the internet.

